

理解度確認テスト：Day22 ～Day28

解答例

問1 解答

(1) **イ** `LibrarySystem is an ArrayList` とは言えない。`has-a` 関係（「持つ」関係）を表すにはコンポジションを使い、`private List<Book> books = new ArrayList<>()` とフィールドで保持するのが正しい設計である。

(2) `private Book book;`（または `Book book;`）

(3) 例) `Book implements Deletable` — 「Book は Deletable（削除可能なもの）の一種だ」と言えるため `is-a` 関係が成立し、継承（インターフェース実装）が適切。

問2 解答

(1) 書籍の貸し出し処理（蔵書の空き確認・`Lending` オブジェクトの生成）は `LibrarySystem` の責任である。`Book` クラスに追加すると「書籍情報の保持」という本来の責任に加えて「業務ロジックの実行」という別の責任を持つことになり、単一責任の原則（1クラス1責任）に違反する。変更理由が増えると保守性が低下する。

(2) **イ** 集約（Aggregation）は「管理する側が削除されても管理される側は独立して存在できる」弱い所有関係を表す。

問3 解答

(1) **イ**（防御的プログラミング）

(2) **イ** `enum` はコンパイラが型を保証するため、存在しない値を代入しようとするとコンパイルエラーになる。`String` で比較する場合はタイプミス（"`AVIALABLE`" 等）が実行時まで気づかれない。

(3) **イ** `final` フィールドは一度コンストラクタで値が設定されると再代入できなくなる（イミュータブル性の向上）。アクセス制御（`private` 等）とは別の概念である。

問4 解答

(1) **イ**（遅延評価 / Lazy Evaluation）

(2)

ステップ	種類	説明
(1)	中間操作	A ：削除済み（ <code>isDeleted() == true</code> ）の書籍を除外する
(2)	中間操作	B ： <code>extractor</code> で取り出した文字列にキーワードが含まれるものに絞り込む
(3)	中間操作	C ：タイトルの昇順で並び替える

ステップ	種類	説明
(4)	D: 終端操作	結果をリストとして収集する
(3)	ウ (メソッドのパラメータ化 / Parameterize Method)	

問5 解答

- (1) イ (各 `@Test` メソッドの実行前に毎回実行される)
- (2) イ テストが同じオブジェクトを共有すると、あるテストの実行後の状態が次のテストに影響する。
`@BeforeEach` で毎回新しいインスタンスを生成することで各テストを独立させる (テストの独立性)。
- (3)
- **Arrange (準備)**: テスト対象のオブジェクトや入力データを準備する。`setUp()` 内の初期化がこれにあたる。
 - **Act (実行)**: テスト対象のメソッドを実際に呼び出す。
 - **Assert (検証)**: 実行結果が期待通りかを検証する。今回は `status` が `LENDING` になっているかを確認する。
- (4) `testLendSuccess` → 正常系 / `testLendAlreadyLending` → 異常系

問6 解答

- (1) 例:

No.	観点	状況 (入力・前提条件)	期待する結果
1	正常系	未延滞・未延長・days=7	返却期限が7日延長される
2	正常系	未延滞・未延長・days=1 (最小値)	返却期限が1日延長される
3	正常系	未延滞・未延長・days=30 (最大値)	返却期限が30日延長される
4	異常系	延滞中 (返却期限が昨日)・days=7	<code>IllegalStateException</code> がスローされる
5	異常系	未延滞・すでに1回延長済み・days=7	<code>IllegalStateException</code> がスローされる
6	境界値	未延滞・未延長・days=0	<code>IllegalStateException</code> がスローされる
7	境界値	未延滞・未延長・days=31	<code>IllegalStateException</code> がスローされる

- (2)

1. テスト対象メソッドの仕様 (前提条件・処理内容・戻り値・例外)
2. 境界値 (最小・最大・ゼロ・null・空文字など) を明示する
3. 正常系・異常系・境界値を分類して列挙するようプロンプトに指示する

問7 解答

(1) ID生成ロジック (`String.format(...)` と カウンタのインクリメント) が3か所に重複している。DRY 原則 (Don't Repeat Yourself) に違反しており、書式を変更する場合に3か所すべてを修正しなければならない。

(2)

```
private String generateId(String prefix, String format, int seq) {
    return prefix + String.format(format, seq);
}

// 呼び出し例
generateId("BK", "%04d", bookIdCounter++) // → "BK0001"
generateId("CP", "%05d", copyIdCounter++) // → "CP00001"
generateId("MB", "%04d", memberIdCounter++) // → "MB0001"
```

(3) ア

ガード節 (早期 return) の適用前は、メインロジックが `if (条件が真なら処理) else (早期 return)` のように、正常系の処理がネストの中に入る構造になっている。

問8 解答

(1)

```
/**
 * 指定された書籍の貸出可能な蔵書を1件返す。
 *
 * @param book 検索対象の書籍
 * @return 貸出可能な蔵書が存在する場合はその蔵書を含む Optional、存在しない場合は空の
Optional
 */
```

(2)

空欄	答え
A (Deletable を実装するクラス)	Book
B (Book への参照を「多対1」で持つクラス)	BookCopy
C (BookCopy が参照する列挙型クラス)	CopyStatus
D (CopyStatus のステレオタイプ)	enumeration

(3) 例 (4項目):

1. プロジェクトの概要・目的
2. 動作環境・前提条件 (Java バージョン等)
3. セットアップ手順 (ビルド・実行方法)

問9 解答

(1)

1. **コンテキスト（背景・コード・エラーメッセージ）をセットで渡している**：AIが問題を正確に把握できるため、的外れな回答が減る。
2. **自分の仮説を示している**：AIに答えを出させるだけでなく、自分の考えを評価させることで、理解の正確さを確認でき、学びが深まる。

(2) イ、エ

(3)

自由記述（採点基準：①具体的なシーンが描かれているか ②効果的・非効果的の自己分析があるか）

問10 解答

(1) **E（リファクタリング）フェーズ** リファクタリングはコードの振る舞いを変えずに内部構造を改善する作業であり、変更によってバグが混入していないかをテストコードで自動検証できることが必須条件となる。テストが「安全網」として機能して初めて安心してリファクタリングに取り組める。

(2) **ア：外部から見た振る舞い（機能） イ：内部構造・品質（可読性・保守性） ウ：既存の動作（テストの全件グリーン）**

(3) 自由記述（採点基準：①Day22～28の学びが具体的に記述されているか ②AI活用に関する気づきや反省が含まれているか）
